

Towards more interpretable models and solutions for conservation decisions

James Lavis

Industry Supervisor: Jonathan Ferrer-Mestres

Academic Supervisor: Marcus Hoerger

April 2023

Abstract

Markov decision processes (MDPs) model decision making under uncertainty and are applied in various domains such as reinforcement learning, computational decision planning, and conservation science. However, the optimal policies of MDPs used in human operated domains are difficult to interpret due to the large number of states required to model the decision environment. In recent work by Ferrer-Mestres et al., 2020, a state abstraction method was proposed where users could specify a maximum number of abstract states and find state abstractions that minimise the performance loss. The result is a smaller K-MDP which is easier for human operators to interpret. The state abstraction functions proposed, however, rely on the optimal policy and optimal value functions of the original MDP, which can be computationally expensive to solve for. This project aims to circumvent the requirement of solving the original MDP before finding state abstractions by using traditional machine learning techniques and state of the art graph neural network approaches. The state abstraction problem is formulated as a multiclass classification problem where the abstract states are treated as classes. Although traditional machine learning techniques maximize classification accuracy, they fail to consider so called sensitive states, which when misclassified significantly impact the effectiveness of the K-MDP. To overcome this limitation, a graph convolutional network is used to take advantage of the graph structure of MDPs, and is able to successfully classify sensitive states, while producing an effective K-MDP.

1 Introduction

Markov decision processes (MDPs) are a convenient mathematical tool for working with systems and problems which require sequential decision making under uncertainty (Puterman, 1994). MDP's are applied in a reinforcement learning context to learn optimal decision-making policies in dynamic environments. By modeling the system as a set of states, actions, and transition probabilities, MDPs allow agents to evaluate and select actions based on maximizing expected rewards or minimizing costs Sutton and Barto, 1992. MDP's have also been applied in conservation studies to help repopulate endangered species, control invasive species, manage fisheries and to test behavioural ecology theories (Marescot et al., 2013). As apposed to the reinforcement learning context, these domains are human operated domains and require a human operator to be able to interpret and make use of the policies computed for MDPs. However, policies computed for MDPs with thousands of states are in practice difficult for human decision makers to interpret. In human operated systems, it is important that policies can be interpreted and explained to guide decision making (Petrik and Luss, 2016).

Explainable artificial intelligence (XAI) is a field of research which aims to allow human operators to understand and trust the results and outputs of machine learning and AI systems. Emerging research on XAI has mostly focused on the explainability of machine learning algorithms (Guidotti et al., 2018), there is recent interest in the explainability of planning solutions, such as those generated by MDPs (Dazeley et al., 2023), though these are focused towards applications in reinforcement learning. Recent work has proposed to improve the interpretability of MDPs by finding a state abstraction.

State abstraction approaches aim to reduce the size of large state spaces by aggregating states that are similar under some metric (Li et al., 2006). Given a state space $S = \{s_1, \dots, s_N\}$ and another state space $S' = \{s'_1, \dots, s'_k\}$ where $k < N$, a state abstraction function is a function $\phi : S \rightarrow S'$ which maps the states in S to an abstract state in S' . MDP solvers can then be applied to MDPs built from the abstract states to compute simpler policies. Of particular interest are state abstraction functions which minimise the loss in performance of the optimal policy of the MDP. Exact state abstraction functions aggregate states that are exactly equal given some metric (Li et al., 2006), whereas approximate state abstraction functions manage to cluster states into abstract states if they are instead similar under some metric. (Abel et al., 2017). Approximate state abstraction functions allow for greater flexibility in grouping states together, as there is no requirement to force the state abstraction metric to a certain value in order for two states to be abstracted into the same abstract state.

In the past, state abstractions have been focused on finding the smallest abstract state space available given a pre-defined loss in performance (Abel et al., 2017). A different approach, however, is to find a state abstraction function that minimises the performance loss given a pre-defined state space size. One proposal to do this is to find a K-MDP (Ferrer-Mestres et al., 2020). Given an MDP M , and a state abstraction function ϕ , a K-MDP M_K is the an MDP with at most K states that minimises the performance loss. This approach provides human operators with an almost equivalent MDP that is much smaller in size and therefore easier to interpret. However, finding a K-MDP is computationally expensive for large MDPs due to the fact that the state abstractions proposed in Ferrer-Mestres et al., 2020 are based on the optimal policy and optimal value functions of the original MDP. The aim of this project is to investigate the use of traditional and state of the art machine learning techniques to circumvent the requirement that the optimal policy and optimal value function of the original MDP needs to be obtained before a K-MDP can be found. This will provide a method for generating state abstractions with less computational requirements, leading to larger MDPs being able to be interpreted and applied more quickly and easily in practice.

The project was split into two phases, each taking around six months. The first phase investigated the use of traditional machine learning techniques by setting the state abstraction problem up as a multiclass classification problem. I investigated different ways of representing the features of the MDP, and investigated the effectiveness of multiple machine learning models. During the second phase I investigated the use of graph convolutional network which could take advantage of the relationships between states in an MDP. A lot of work was done to write the state abstraction algorithms from Ferrer-Mestres et al., 2020 and the algorithms to solve an MDP. Section 2 of this report focuses on the background information used during this project, and reviews the essential information on MDPs, state abstractions, K-MDPs, deep learning and graph neural networks. Section 3 discusses briefly the implementation of the algorithms and the steps required to evaluate the model output by building a K-MDP with the result. Section 4 focuses on the results of using traditional machine learning approaches and Section 5 focuses in the application of the graph convolutional network.

2 Background

2.1 Markov Decision Processes

Markov Decision Processes, or MDPs, model a situation where an agent acts in an environment and gains some notion of reward for its actions. Formally, a MDP is mathematically defined as a tuple $\langle S, A, T, R, \gamma \rangle$ where

- $S = \{s_1, \dots, s_N\}$ is a set of N states which represents all possible configurations of the environment that the agent might observe
- $A = \{a_1, \dots, a_M\}$ is a set of M actions which the agent can take in any state
- $T : S \times A \rightarrow \Pi(S)$ is a function, where $\Pi(S)$ is a probability distribution over S . $T(s_i, a_j, s_k)$ represents

the probability of the system transitioning from s_i to s_k when the agent takes action a_j while in state s_i , and is defined such that $\sum_{k=1,\dots,N} T(s_i, a_j, s_k) = 1$ for $i \in \{1, \dots, N\}$ and $j \in \{1, \dots, M\}$.

- $R : S \times A \rightarrow \mathbb{R}$ is a function where $R(s_i, a_j)$ is the reward given to the agent for taking action a_j while in state s_i
- $\gamma \in (0, 1)$ is a discount factor.

Markov decision processes obey the Markov property, where the probability distribution of the next state depends only on the current state and the current action and does not depend on the previous state of the system and the previous actions taken by the agent (Kaelbling et al., 1998).

Now, let $s(t) = s_t$ be the state that the agent occupies at time t , and let $a(t) = a_t$ be the action that the agent takes at time t . The aim of using this model would be to allow the agent to act in a way that maximises some notion of long-run reward. If the finite horizon case, the agent has T time-steps to act in its environment and accumulate rewards and should act to maximise its expected reward over time,

$$\mathbb{E} \left[\sum_{t=0}^T R(s_t, a_t) \right]. \quad (1)$$

While this formulation can be inconvenient in most cases as it is difficult to determine an appropriate value for T , and it does not deal with terminating states, which are states that, when reached, have no transitions away from it. A more convenient approach is to assume that the agent has an infinite lifetime and discount the value of far off expected returns. For the infinite horizon discounted model, we define the discount factor $0 \leq \gamma \leq 1$ to discount future rewards and set the model to maximise the following;

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]. \quad (2)$$

The discount factor ensures that the sum converges and that the agent is more focused on accumulating immediate rewards over far off rewards. The infinite horizon model also deals with the problem of terminating states as the agent does not need to account for that possibility occurring in finite time (Kaelbling et al., 1998).

Now, we define a policy $\pi : S \rightarrow A$ as a mapping from the set of states to the actions that define what action is to be taken when the agent is in any particular state. A policy is a stationary policy if the policy does not change over time. That is, for all $s \in S$ and $a \in A$, if $\pi_t(s) = a$ then $\pi_{t+1}(s) = a$ for all $t \geq 0$. Non-stationary policies are out of the scope of this project.

We can then define the value function at each state under the stationary policy π as

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s \right]. \quad (3)$$

This expression can be reformulated as

$$\begin{aligned} V^\pi(s) &= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s \right] \\ &= R(s_0, \pi(s_0)) + \sum_{s' \in S} T(s_0, \pi(s_0), s') \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_1 = s' \right] \\ &= R(s_0, \pi(s_0)) + \gamma \sum_{s' \in S} T(s_0, \pi(s_0), s') \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, \pi(s_t)) \mid s_1 = s' \right] \\ &= R(s_0, \pi(s_0)) + \gamma \sum_{s' \in S} T(s_0, \pi(s_0), s') V^\pi(s') \end{aligned}$$

where the last line uses the definition of the value function and takes advantage of the infinity in the summation. Therefore, for a given policy π , the value function under that policy can be computed iteratively as

$$V^\pi(s) = R(s_0, \pi(s_0)) + \gamma \sum_{s' \in S} T(s_0, \pi(s_0), s') V^\pi(s'). \quad (4)$$

2.1.1 Solving MDP's

Many problems that can be modelled using an MDP are focused on finding a policy that maximises the value function at each state. This policy is called the optimal policy and can be defined formally as

$$\pi^*(s) = \arg \max_{a \in A} \left[R(s, a) + \sum_{s' \in S} T(s, a, s') V^*(s') \right], \quad (5)$$

where V^* is the corresponding value function. However, this definition is unclear as the optimal policy is defined based on the optimal value function, which is also defined in terms of the optimal policy. The optimal value function and the optimal policy can, however, be solved using a variety of algorithms. Here, we consider the value iteration algorithm.

The value iteration algorithm is defined for MDPs in the case where the state and action space is finite and is defined in algorithm 1 (Kaelbling et al., 1998).

Algorithm 1 Value Iteration Algorithm

```

1:  $V_1(s) = 0 \quad \forall s$ 
2:  $t = 1$ 
3: repeat
4:    $t = t + 1$ 
5:   loop for all  $s \in S$ 
6:     loop for all  $a \in A$ 
7:        $Q(s, a) = R(s, a) + \gamma \sum_{s' \in S} T(s, a, s') V_{t-1}(s')$ 
8:     end loop
9:      $V_t(s) = \max_a Q(s, a)$ 
10:  end loop
11: until  $|V_t(s) - V_{t-1}(s)| < \epsilon$  for all  $s \in S$ 

```

Value iteration works by computing a sequence of finite horizon optimal value functions. The first iteration sets the value function to the maximum reward for that state. The second iteration is equivalent to a finite horizon MDP with $T = 2$ and discounting the future step. The third iteration is equivalent to a finite horizon with $T=3$ and discounting the future steps, and so on. As the algorithm progresses, the computed optimal value function discounts the earlier steps by γ and approaches the optimal value function that would be found in the infinite horizon case. The algorithm terminates when the difference between the computed value function and the computed value function in the previous step is less than ϵ . It can be shown in Puterman, 1994 that if $|V_t(s) - V_{t-1}(s)| < \epsilon$ for all $s \in S$, then

$$\max_{s \in S} |V_\pi(s) - V^*(s)| < \frac{2\epsilon\gamma}{1-\gamma}. \quad (6)$$

That is, when the terminal condition of the algorithm is met, the value function found by the algorithm is approximately the true optimal value function, and the policy computed by the algorithm is close to the optimal policy (Williams and Baird, 1993). In fact, the policy computed by the algorithm approaches the optimal policy long before the terminating condition is met (Puterman, 1994).

2.2 K-MDP's

2.2.1 K-MDP definition

Formally, a K-MDP can be defined as follows (Ferrer-Mestres et al., 2020). Given an MDP $M = \langle S, A, T, R, \gamma \rangle$, a K-MDP is a tuple $M_K = \langle S_K, A, T_K, R_K, \gamma, \phi \rangle$ where

- $\phi : S \rightarrow S_K$ is the state abstraction function.
- $S_K = \{\phi(s) \mid s \in S\}$ is the abstract state space derived by applying the state abstraction function on S
- A is the same action space as the original MDP M
- $T_K : S_K \times A \rightarrow \Pi(S_K)$ where $\Pi(S_K)$ is a probability distribution over S_K . $T(s_K, a, s'_K)$ represents the probability of transitioning to s'_K given the agent is in s_K and has taken action a .
- $R_K : S \times A \rightarrow [0, R_{\max}]$ is the reward functions for the abstract states
- $\gamma \in (0, 1]$ is a discount factor.

Given a state abstraction function ϕ , the transition functions for the K-MDP are calculated to be

$$T_K(s_K, a, s'_K) = \sum_{s \in \phi^{-1}(s_K)} \sum_{s' \in \phi^{-1}(s'_K)} T(s, a, s') \omega(s) \quad (7)$$

where the inverse state abstraction function is defined to be $\phi^{-1}(s_K) = \{s \in S \mid \phi(s) = s_K\}$ and $\omega(s)$ defines a distribution over states such that

$$\forall s_K \in S_K, \quad \sum_{s \in \phi^{-1}(s_K)} \omega(s) = 1. \quad (8)$$

Similarly, the reward functions for the K-MDP are calculated as

$$R_K(s_K, a) = \sum_{s \in \phi^{-1}(s_K)} R(s, a) \omega(s). \quad (9)$$

2.2.2 Abstraction Performance

The optimal solution for a K-MDP is a policy $\pi_K^* : S_K \rightarrow A$ that maximises the value function for the K-MDP. This policy can then be applied to the original MDP by using the mapping function ϕ to map the original state s to its abstract state s_K in order to extract the action under π_K^* . The associated value function is

$$V_\phi^{\pi_K^*}(s) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R(s, \pi_K^*(\phi(s))) \mid s_0 = s \right]. \quad (10)$$

The aim of finding a K-MDP is to find the reduced state space S_K where $|S_K| \leq K$ that minimises the gap between the optimal value function of the MDP and the value function of the MDP under the K-MDP optimal policy (Ferrer-Mestres et al., 2020). This gap minimisation problem can be expressed as

$$gap^* = \min_{S_K \in P(S); |S_K| \leq K} \max_{s \in S} [V^*(s) - V_\phi^{\pi_K^*}(s)]. \quad (11)$$

While this expression clearly defines the problem of finding a set of abstract states that minimises the gap, it can be reformulated as a way to measure the performance of the state abstraction ϕ . Define the gap as

$$gap = \max_{s \in S} [V^*(s) - V_\phi^{\pi_K^*}(s)]. \quad (12)$$

Since the optimal value function $V^*(s) \geq V^\pi(s)$ for all possible policies π , we can define the error as

$$\text{error} = \frac{\max_{s \in S} [V^*(s) - V_{\phi}^{\pi_K^*}(s)]}{\max_{s \in S} V^*(s)} \quad (13)$$

$$= \frac{\text{gap}}{\max_{s \in S} V^*(s)}. \quad (14)$$

Since $\text{error} \in [0, 1]$, an error close to one represents a state abstraction that performs poorly on the MDP, whereas a error close to zero represents a state abstraction which performs well by producing a policy that performs well on the MDP.

2.2.3 Build K-MDP algorithm

Given a state abstraction function, K-MDPs can be built from the abstract states using the BUILD-K-MDP algorithm. The algorithm takes in an MDP, a set of abstract states and the state abstraction function and outputs a K-MDP (Ferrer-Mestres et al., 2020).

Algorithm 2 Build-K-MDP

Require: $M = \langle S, A, T, R, \gamma \rangle, S_K, \phi$

```

1: for  $s_K \in S_K$  do
2:   for  $s \in \phi^{-1}(s_K)$  do
3:      $\omega(s) = \text{computeWeights}(\phi, s_K)$ 
4:   end for
5:   for  $a \in A$  do
6:      $R_K(s_K, a) = \sum_{s \in \phi^{-1}(s_K)} R(s, a) \omega(s)$ 
7:     for  $s'_K \in S_K$  do
8:        $T(s_K, a, s'_K) = \sum_{s \in \phi^{-1}(s_K)} \sum_{s' \in \phi^{-1}(s'_K)} T(s, a, s') \omega(s)$ 
9:     end for
10:   end for
11: end for
12:  $M_K = \langle S_K, A, T_K, R_K, \gamma \rangle$ 

```

$\omega(s)$ represents a probability distribution over the states $s \in \phi^{-1}(s_K)$. In our case, we assume a uniform distribution over the states so that $\text{computeWeights}(\phi, s_K) = 1/|\phi^{-1}(s_K)|$ for all $s \in \phi^{-1}(s_K)$.

2.2.4 $\phi_{a_d^*}$ State Abstraction Function

There were a few state abstraction functions introduced in Ferrer-Mestres et al, 2020, with one that significantly outperformed the others. The $\phi_{a_d^*}$ is motivated by the exact state abstraction function ϕ_{a^*} from (Li, Walsh and Witman, 2006) which groups states together as follows:

$$\phi_{a^*}(s_i) = \phi_{a^*}(s_j) \implies (\pi^*(s_i) = \pi^*(s_j)) \wedge (V^*(s_i) = V^*(s_j)). \quad (15)$$

This state abstraction function preserves the optimal actions and their values (Li et al., 2006) but is too strict in its definition. The $\phi_{a_d^*}$ state abstraction relaxes the condition that the optimal value functions need to be equal and rather defines a parameter d , which controls how close together the value functions can be in the same abstract state. The $\phi_{a_d^*}$ state abstraction groups together states as follows:

$$\phi_{a_d^*}(s_i) = \phi_{a_d^*}(s_j) \implies (\pi^*(s_i) = \pi^*(s_j)) \wedge \left\lceil \frac{V^*(s_i)}{d} \right\rceil = \left\lceil \frac{V^*(s_j)}{d} \right\rceil \quad (16)$$

where $0 < d < V_{\max}^*$, V^* is the optimal value function for the solved MDP and π^* is the optimal policy. Intuitively, because $\phi_{a_d^*}$ preserves the optimal actions, it is more likely to minimise the gap. This advantage, however, comes at the cost of not being feasible when the number of unique optimal actions in π^* is greater

than K . Under this state abstraction, the value loss from state abstraction can be found to be (Ferrer-Mestres et al., 2020)

$$V^*(s) - V_{\phi_{a_d^*}^{\pi^*}}(s) \leq \frac{2dR_{\max}}{(1-\gamma)^2} \quad \forall s \in S. \quad (17)$$

Under this state abstraction function, states are grouped together optimally by performing a binary search over d while the number of abstract states $|S_K|$ is less than K . This is shown in algorithm 3 Here, *bindings* is

Algorithm 3 $\phi_{a_d^*}$ Algorithm

Require: $M, K \geq 1, p_{target}, V^*, \pi^*$

```

1:  $d^+ = V_{\max}$ 
2:  $d^- = 0$ 
3: repeat
4:    $p = d^+ - d^-$ 
5:    $d = d^- + \frac{d^+ - d^-}{2}$ 
6:   for  $s \in S$  do
7:      $a_s^* = \pi^*(s)$ 
8:      $bindings = [\lceil V^*(s)/d \rceil, a_s^*]$ 
9:      $S_K = unique(bindings)$ 
10:    if  $|S_K| \leq K$  then
11:       $d^+ = d$ 
12:    else
13:       $d^- = d$ 
14:    end if
15:  end for
16: until  $p < p_{target}$ 
17:  $M_K = \text{Build-K-MDP}(M, S_K, \phi)$ 
```

a data structure that stores the ceiling of $V^*(s)/d$ and the optimal action a^* . It can be shown that algorithm 3 solves the K-MDP problem within a precision target of p_{target} in a finite number of iterations if K is greater than or equal to the number of unique actions of the original MDP optimal policy π^* .

2.3 Machine Learning

2.4 Graph Convolutional Networks

2.4.1 Deep Learning

In the simplest case, a deep neural network is a composition of parameterised linear functions f^l for $l \in \{1, \dots, L\}$, each followed by an element-wise non-linear activation σ (Kipf, 2020).

$$NN_{\theta} = f^L \circ \sigma \circ \dots \circ f^2 \circ \sigma \circ f^1. \quad (18)$$

Neural networks can be used as function approximates with learnable parameters θ . The simplest neural network architecture is a multilayer perceptron, MLP where f^l is defined to be (Rosenblatt, 1961)

$$f^l(h) = W_l h + b_l \quad (19)$$

where W_l is a matrix of learnable weights for the l th layer, b_l is a learnable bias vector, and h_l represents the feature vectors (or hidden representations) of the input data. For the activation function σ , a common choice is the rectified linear unit, or ReLU function, defined as

$$ReLU(h) = \max(0, h). \quad (20)$$

Neural networks are trained using the backpropagation algorithm and variants of the stochastic gradient descent algorithm to minimise a loss function L . The loss function L is typically chosen to guide the network

towards a specific output. In the case of multiclass classification with C classes, the output layer of the neural network will have C outputs, one for each class. The outputs are then passed through a softmax function which estimates the probability distribution over each class. The softmax is defined as

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (21)$$

where z_i represents the neural network output for class i . After estimating the probabilities using the softmax function, the loss function is typically chosen to be the cross-entropy loss, defined to be

$$\text{CrossEntropyLoss}(y, \hat{y}) = - \sum_{c=1}^C y_c \log(p_c), \quad (22)$$

where y_c is a binary indicator indicating the true class of the sample, and p_c is the output softmax probability of the sample belonging to that class (Hastie et al., 2009).

2.4.2 Graph Convolutional Networks

Graph neural networks are a neural network architecture built on top of graph structures. It takes in a graph $G = (V, E)$, where V is a set of nodes $\{v_1, \dots, v_N\}$ and E is a set of edges (v_i, v_j) that connect the nodes v_i and v_j . If for all i, j , $(v_i, v_j) \in E$ and $(v_j, v_i) \in E$, we say that the graph is undirected. Associated with each node is a d -dimensional feature vector x_i , where d is the number of features for each node. Optionally each edge can have an associated feature vector $x_{(i,j)}$.

The structure of the graph then determines the message-passing updates, which are executed in sequence to obtain updated node representations and edge representations. Let h_i^l be the node representation of node v_i on the l^{th} iteration, and let $h_{(i,j)}^l$ be the edge representation after the l^{th} update. Set $h_i^0 = x_i$ and $h_{(i,j)}^0 = x_{(i,j)}$ (Kipf, 2020).

$$h_{(i,j)}^{l+1} = f_{\text{edge}}(h_i^l, h_j^l, x_{(i,j)}) \quad (23)$$

$$h_i^{l+1} = f_{\text{node}}(h_i, \sum_{h_j \in N(v_i)} h_{(j,i)}^l, x_i) \quad (24)$$

where $N(v_i)$ is the neighbourhood of v_i , that is all the nodes v_j that v_i is connected to. The update to the network is called a message-passing operation. Since each message-passing operation collects information from each of the neighbours, more message-passing operations aggregate information from further along the graph.

For a graph convolutional network, the message passing is done through the following function (Kipf and Welling, 2017):

$$h_i^{(l+1)} = \sigma \left(W_0^{(l)T} h_i^{(l)} + \sum_{v_i \in N(v_i)} c_{i,j} W_1^{(l)T} h_j^{(l)} \right) \quad (25)$$

where σ is a non-linear activation function such as the element-wise ReLU function, $W_0^{(l)}$ and $W_1^{(l)}$ are learnable $d_l \times d_{d+1}$ parameter matrices and $c_{i,j} = 1/\sqrt{D_{i,i}D_{j,j}}$ is a normalisation constant, where $D_{i,i}$ is the degree of node v_i . The final node representations h^L can then be used for regression or node classification problems in the normal way described above.

3 Algorithm Implementations

The aim of this project is to use machine learning techniques to learn the state abstraction functions from the MDP without the need to solve for the optimal value function and optimal policy first. The project was completed in two phases. The first phase used traditional supervised learning approaches and treated the

problem as a multiclass classification problem. There were three datasets used in this phase, each representing an MDP, and for each dataset a set of models were trained and tested using cross validation. The second phase of the project involved using graph convolutional networks to take advantage of the graph structure of the MDP.

For each model tested, the MDP's used need to be generated and processed before the model is applied, and then the model outputs need to be evaluated. MDP's for the first phase of the project are generated within CSIRO and given as csv files with the columns representing the transition probabilities, reward functions and the abstract state assigned by the $\phi_{a_d}^*$ state abstraction function, and the rows representing the states of the MDP. For the second phase of the project the MDP's are generated via a numerical simulation.

For each MDP, the following steps are applied:

1. Solve MDP using value iteration algorithm (algorithm 1)
2. Calculate state abstractions using $\phi_{a_d}^*$ state abstraction (algorithm 3)
3. Train the machine learning model
4. Predict abstract states using the model
5. Build a K-MDP from the predictions using algorithm 2
6. Solve the K-MDP using the value iteration algorithm
7. Map the actions from the K-MDP optimal policy to the original MDP
8. Solve for the value functions when following the K-policy
9. Calculate the gap and the error using equations 12 and 14

Each MDP is stored in a python class where the transition functions and reward functions can be easily accessed, and each algorithm above is implemented separately in different python functions. Algorithm 1 is implemented using the MDPToolBox python package, a python version of matlab code designed to work with MDPs (Chadès et al., 2014). Algorithm 2 and algorithm 3 are implemented in an unreleased python toolbox created within CSIRO, and function exactly as the pseudo code implies. Lastly, to cut down on computation time, the value functions with respect to the k-policy takes advantage of the computational speed up of Numpy's matrix multiplication implementation, by recognising the fact that under policy π , R^π is a column vector and T^π is a matrix of transition probabilities, meaning that the value function V^π can be defined recursively as a matrix equation (Harris et al., 2020)

$$V^\pi = R^\pi + \gamma T^\pi V^\pi. \quad (26)$$

4 Traditional Machine Learning Approach

The datasets for the first phase are derived from the following problems from computational sustainability studies:

1. Sea Otter and Northern Abalone (SONA) Problem: The sea otters of northern Canada and their main prey, the northern abalone, are both on the endangered species list. This is a classic problem described by (Chadès et al., 2012). The objective here is for managers to take actions that maximise the abundance of both species. The MDP has 819 states representing different configurations of sea otter populations and densities of northern abalones. Managers can choose from four actions: Introduce sea otters, take antipoaching measures, control the sea otter population, and one half antipoaching and one half sea otter population control. In the model, antipoaching reduces the illegal harvesting of abalone by 50%, control of the sea otters remove the number of otters above 60% of their carrying capacity. Half antipoaching and half control of sea otters does both, but at half the level of effectiveness.

2. Reserve Problem: Introduced in (Costello and Polasky, 2004), this model is used for dynamic site reservation. There are J sites and I species. At any given time period, any site can be in one of three states: *developed*, *reserved* or *unreserved*. A species can only exist on a site that site has not developed. At each time t , it is possible to choose one unreserved site for reservation, changing the state from *unreserved* to *reserved*. Also, at each time t , an *unreserved* site j can become developed at the end of the period with a probability p_j . For a problem with J sites, there are 3^J possible states and J possible actions, corresponding to choosing a reserve on each site. The objective of the reserve selection problem is to minimise the species lost due to development (Sabbadin et al., 2007).

Three separate MDPs were used to test the viability of traditional machine learning approaches. The first is the SONA problem, with 819 states and 4 actions, the second is the reserve problem where $J = 6$, with 729 states and 6 actions; and the third is the reserve problem where $J = 7$, with 2187 states and 7 actions. Each of the problems are represented in tabular form with the rows of the table representing each of the states in the MDP, and the columns of the table representing the transition functions under each action, the reward functions under each action and the label of the abstract state in the K-MDP that each state belongs to. Each of the datasets are treated separately, and a multiclass classifier is trained on each.

4.0.1 Feature Engineering

Feature engineering is used to improve the performance of the classifiers. The first feature set was the original MDP representation, where columns encode the transition functions and reward functions for each state. This is a logical choice as all the information about the MDP is encoded here, and is also used as a baseline. The second feature engineering transformation is based on the idea that the transition probabilities determined the dynamics of the system. Thus, for each state, the most probable transitions were listed, with the first feature being the most probable transition, the second feature being the second most probable transition, and so on. These features were then one-hot-encoded as they represented categorical data in the form of a label for the transitions, and were used in conjunction with the reward functions. The number of most probable transitions could be set during hyperparameter tuning, but was instead taken to be ten to keep the dimension of the dataset low. The third feature transformation was derived by noticing that the $\phi_{a_d^*}$ state abstraction function was based on two things: the optimal actions are the same, and the value functions are similar. For two states s_α and s_β , when the value functions are similar, then

$$(V^*(s_\alpha) - V^*(s_\beta))^2 \leq \epsilon$$

for some value of ϵ . Now, using the definition of the value function in equation 4, the square difference can be expanded on.

$$\begin{aligned} (V^*(s_\alpha) - V^*(s_\beta))^2 &= \left[R(s_\alpha, \pi^*(s_\alpha)) + \gamma \sum_{s' \in S} T(s_\alpha, \pi^*(s_\alpha), s') V^*(s') - R(s_\beta, \pi^*(s_\beta)) - \gamma \sum_{s' \in S} T(s_\beta, \pi^*(s_\beta), s') V^*(s') \right]^2 \\ &= \left[R(s_\alpha, \pi^*(s_\alpha)) - R(s_\beta, \pi^*(s_\beta)) + \gamma \sum_{s' \in S} (T(s_\alpha, \pi^*(s_\alpha), s') - T(s_\beta, \pi^*(s_\beta), s')) V^*(s') \right]^2 \\ &= [R(s_\alpha, \pi^*(s_\alpha)) - R(s_\beta, \pi^*(s_\beta))]^2 \\ &\quad + 2\gamma (R(s_\alpha, \pi^*(s_\alpha)) - R(s_\beta, \pi^*(s_\beta))) \sum_{s' \in S} (T(s_\alpha, \pi^*(s_\alpha), s') - T(s_\beta, \pi^*(s_\beta), s')) V^*(s') \\ &\quad + \gamma^2 \left[\sum_{s' \in S} (T(s_\alpha, \pi^*(s_\alpha), s') - T(s_\beta, \pi^*(s_\beta), s')) V^*(s') \right]^2 \\ &\approx [R(s_\alpha, \pi^*(s_\alpha)) - R(s_\beta, \pi^*(s_\beta))]^2. \end{aligned}$$

The approximation is made assuming that in general the squared difference in the reward functions are greater than the terms with squared probabilities, squared γ and γ multiplied by a probability. Therefore, since for larger differences the squared difference in the rewards is the largest term in the expression, it should be a strong indicator of whether two states should be grouped together. The third feature engineering

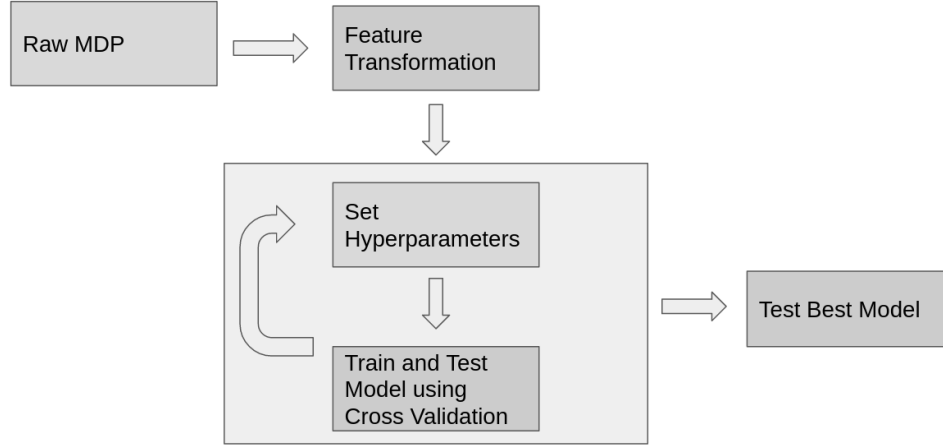


Figure 1: Diagram of the training loop for each model and each dataset. Each dataset has the relevant feature transformation applied to it, and is then split into a train, test and validation set. The train and test sets are used for hyperparameter tuning for each classifier. The validation set is then used to evaluate the classifier’s with the best hyperparameters.

transformation is then derived by finding the square difference in rewards between each state under the same action, and concatenating the transition probabilities for each state and action. This creates $A \times S - A$ new features that can be used, where S is the number of states in the MDP and A is the number of actions.

4.0.2 Model selection and training

The following models are chosen to be tested as a multiclass classifier:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- k-Nearest neighbours
- Multilayer Perceptron

Each model is implemented in the Python Scikit Learn library (Pedregosa et al., 2011). In each case, the inputs to the models are the derived features from the feature engineering step and the outputs are the class label that each state belongs to. Each dataset is randomly split into a train, test and validation set, where each set consists of randomly chosen states. The states in the train and test set are used for hyperparameter tuning using the Optuna module in python (Akiba et al., 2019). The hyperparameters to be tuned are set beforehand and depend on the model that is being trained. The Optuna library heuristically chooses hyperparameters which maximise some objective function, which in this case is chosen to be the five fold cross validated accuracy score. The training loop is detailed in figure 1. While the accuracy score is used to evaluate the model performance as a classifier, the gap function defined in equation 12 is used to measure the performance of the classifiers as state abstraction functions.

4.0.3 Results

Figure 2 shows the accuracy score of each model after hyperparameter tuning and cross validation. The accuracy score is the mean of the accuracy scores calculated under five fold cross validation. For the sea otter and northern abalone problem, the models are able to learn the structure of the MDP well, with the best model being a multilayer perceptron classifier with a single hidden layer of size 100 neurons with ReLU

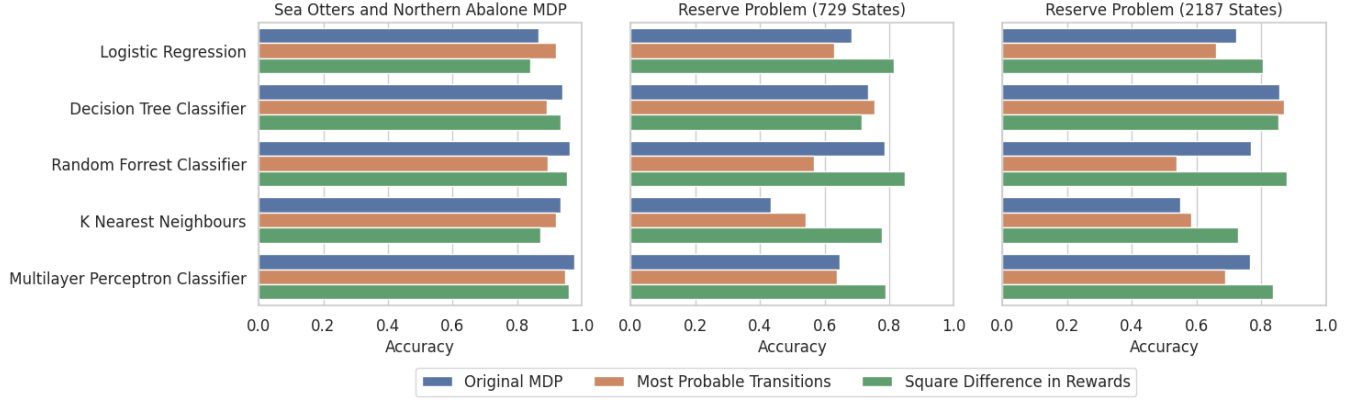


Figure 2: Accuracy score of each model and each feature transform after hyperparameter tuning for each MDP considered. The sea otter and northern abalone MDP performs well across each model and transform, whereas the reserve problem MDPs require more sophisticated approaches.

activation’s. The model works best with the original representation of the MDP with an accuracy of accuracy of this model is 0.976. The output of the model is used as the abstract states, and a k-MDP is created using algorithm 2. The K-MDP is solved using algorithm 1 and the optimal policy from the k-mdp are extracted. Finally, the optimal policy from the k-MDP is mapped back to a policy for the original MDP and is used to determine the value function of the mdp under the k-MDP optimal policy. Using equation 12, the gap for this MDP is found to be 226.14 and the error rate is 0.957. The abstract states calculated using the state abstraction function in algorithm 3 yields a gap of 2.90 and an error of 0.01.

For the reserve problem with 729 states, the models don’t fit as well as they do for the sea otter and northern abalone problem. This is because the reserve problem’s transition probabilities are a lot more sparse than the sea otter and northern abalone problem. For each case except when using decision trees, the best feature transform was to find the square difference in the reward functions, demonstrating here that the value of the reward functions are important for this problem. The best performing model in this case is a random forest model which yields a cross validated accuracy of 0.85. The gap however in this case is 1.41 and the error rate is 0.95. The gap and error rate calculated from the ground truth abstract states in this case are 0.016 and 0.011 respectively.

Finally, for the larger reserve problem, the square difference in the reward functions is also the best feature transformation across each model except for the decision tree classifier, and the best performing model here is the random forest with an accuracy of 0.88. The gap and error rate from this model on the dataset is 1.086 and 0.996 respectively, compared to the ground truth values of 0.018 and 0.016 respectively.

4.0.4 Discussion

In each case, the machine learning models were optimised by hyperparameter tuning using Optuna’s heuristic algorithm, cross validated using 5 fold cross validation and tested on a holdout set. While each model achieved high accuracy scores, the all failed as state abstraction functions with gaps orders of magnitude greater than the $\phi_{a_d^*}$ state abstraction function which the models are trained on. The reason for this may be that there exist states in each MDP that, when misclassified, contribute to a small loss in accuracy, but have a large loss in the MDP performance. Since misclassifying a state into a different abstract state will likely change the optimal action resulting from the K-MDP, an action that is detrimental to the system could be chosen when the system is in such a state. For example, with the sea otter and northern abalone problem, misclassifying a state where one species is near extinction into an abstract state that gets assigned an action to cull that species would result in a detrimental loss to the system. I have decided to term these states ‘sensitive states’ due to their sensitive nature.

It is clear that MDPs can't be treated well with traditional machine learning techniques because states have different levels of importance which cannot easily be captured before modelling without some domain expertise. Since machine learning techniques treat all points with equal importance, even though they weight the classes, they cannot adequately account for 'sensitive states'. Sensitive states from underrepresented classes may be the cause of these problems, as these are the most likely to be misclassified. However, weighting the classes during training may just cause sensitive states in other classes to be more likely to be misclassified as well, and therefore does not solve the problem.

5 Graph Neural Network Approach

States in an MDP are characterised by rewards assigned to taking certain actions, and the probability of transitions to each other state under that action. In that sense, states in an MDP are partly characterised by their relation to other states in the MDP, similar to nodes in a graph. The aim of using a graph neural network architecture is to take advantage of these relationships to construct node classifications which correspond to the abstract states. This approach should be able to account for sensitive states as that information is embedded into the structure of the graph. The graph convolutional network described in section 2.4.2 is chosen because of the way that it aggregates node features, and use the implementation built in the PyTorch Geometric python package (Fey and Lenssen, 2019).

5.1 Dataset generation and representation

MDP's can be represented as graphs by expressing both the states and the actions for each state as nodes, however this representation is not particularly useful for a node classification problem because the majority of nodes will represent actions instead of states, and will not have the same features assigned to nodes that represent states. An alternative representation is to represent the states as nodes in a graph and set the node features to be the reward functions for each action for that state and the transition probabilities to each state under each action. Directed edges are defined between nodes if all of the transition probabilities from one node to another under each action is greater than zero, meaning that there exists some relationship between the states.

The graph neural network is trained on MDP's generated by simulating the conservation reserve problem described earlier. Variation between each MDP is introduced by random starting conditions, and a controllable parameter which sets the probability that a wildfire can occur on the undeveloped sites. One hundred MDP's with $J = 4$ sites and $I = 20$ species are generated (corresponding to 81 states each and 4 actions), with randomness introduced in the starting conditions and probability of wildfires occurring being set to a random value $0 \leq p \leq 1$. This randomness generates substantially different MDP's modelling the same process, meaning that the trained model can be applied to any MDP generated using this process.

5.2 Hyperparameters

The graph convolutional network is set up as a node classification problem with the loss function chosen to be the cross entropy loss defined in equation 22. This loss function is chosen as it is the most relevant for multiclass classification problems and will direct the neural network to maximise the accuracy, precision and recall of the classifier. The implementation used is the one defined in the python PyTorch Library (Paszke et al., 2019).

The one hundred MDP's are split randomly into a training set, a testing set and a validation set with a 50%/25%/25% split respectively. Hyperparameter tuning is performed on the training and testing set, while the validation set is used as a hold out to evaluate the final model. Hyperparameter tuning is done again using the Optuna python package, which does a heuristic search through the hyperparameters in order to maximise a user defined score.

The hyperparameter tuning loop is set up to train the model for up to 500 epochs, or until the cross

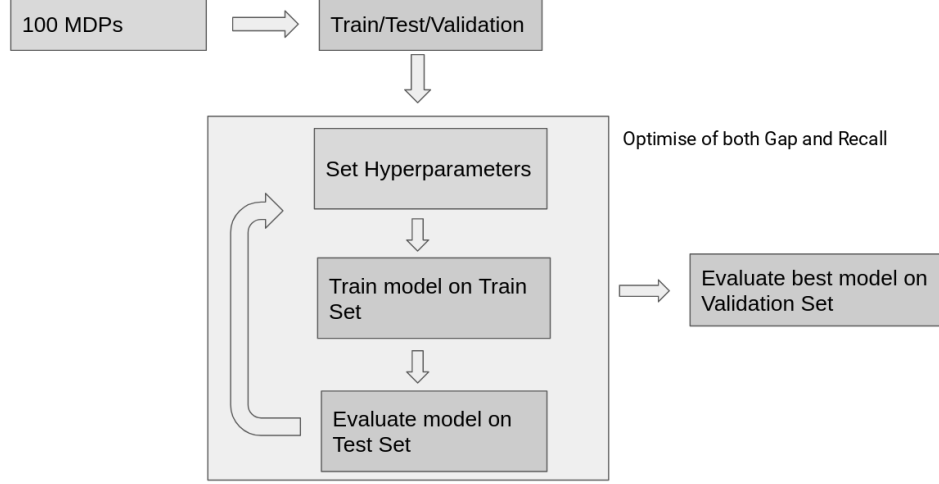


Figure 3: Training Loop for the Graph Convolutional Network. MDP’s are split into train, test and validation sets, and hyperparameter tuning is done on the train and test sets to maximise the recall score and minimise the gap.

entropy loss function converges within a specified bound, and evaluate the quality of the model before updating the hyperparameters. For each epoch, the graph convolutional network (GCN) model is applied to each MDP in the training set, represented as a graph, and the cross entropy loss between the model outputs and the labels assigned by the $\phi_{a_d^*}$ state abstraction function is calculated. Once the GCN has been applied to each MDP in the training set, the cross entropy loss for each is averaged and the weights of the model are updated via ADAM, a variation of stochastic gradient descent. By averaging the cross entropy loss over the training set before updating the weights, the training is stabilised by reducing the noise in cross entropy scores which might arise if the model is updated for each MDP, or a smaller batch size of MDP’s. Once the model training is completed, the model is applied to the test set and the error score is calculated, as well as the model recall score. The error and recall is then averaged over each MDP in the test set. The recall score is used to measure the rate of false negatives. The hyperparameter tuning loop is set up to simultaneously maximise the recall score and minimise the error.

There are a number of hyperparameters which can be chosen which will affect the quality of the model after the training is complete. First, for the cross entropy loss function, we are able to assign a weight to each class so that different classes can be assigned more importance. This is usually to combat the affect of an imbalanced dataset on the classifiers performance. We do not know beforehand how important the class imbalance is to building the model, so I want this to be decided during hyperparameter tuning. Let ω_i be the fraction of samples across all of the MDP’s in the training set that belong to class i . The class weight W_i assigned to the class i in the cross entropy loss is calculated as

$$W_i = \frac{\omega_i^{-\frac{1}{M}}}{\sum_j W_j}, \quad (27)$$

where M is a hyperparameter. The idea behind numerator is that more weight should be assigned to classes that have less representation in the dataset, and the denominator is a normalisation constant. The M hyperparameter is a positive integer and controls how relevant the class imbalance is to the model. As M increased, the weights become more equal and approach $1/C$, where C is the number of classes, and the class imbalance becomes less important for the model.

The deep learning component of the model is trained using the ADAM optimizer, which is an adaptation of the traditional stochastic gradient decent algorithm with many improvements, such as an adaptive learning rate and a momentum calculation which allows the algorithm to converge faster (Kingma and Ba,

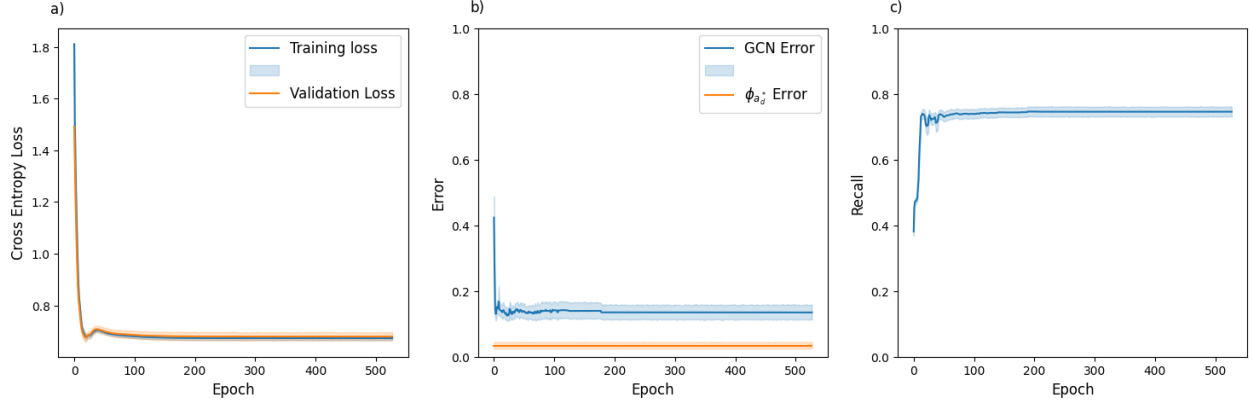


Figure 4: Results of the Graph Convolutional Network applied to the validation set. a) The mean cross entropy loss of the combined training and testing set and the validation set over 500 epochs of training. The shaded region represents the 95% confidence interval b) The predicted mean error of the GCN applied to the validation set, and the original $\phi_{a_d}^*$ state abstraction applied to the same set. The shaded regions represent the 95% confidence interval about the average. c) The mean recall score of the GCN model applied to the validation set over the training lifetime of the GCN. The shaded region is the 95% confidence interval about the mean.

2017). The ADAM optimizer has a few hyperparameters that can be tuned to improve the performance of the deep learning process. The learning rate controls the size of the step during each update to the parameters, and essentially controls the speed at which the algorithm converges. It is important to find the correct value for this parameter as a large learning rate could cause the weights to oscillate around the optimal value without reaching it, and a small value could cause a slow learning process and a convergence to a local minimum instead of the global minimum of the loss function. The weight decay parameter adds a penalty of $\lambda\theta$ for parameters becoming too large, where λ is the weight decay parameter and θ are the model weights (Loshchilov and Hutter, 2019). By regularizing the parameters this way, ADAM prevents the deep learning model from over fitting.

Finally, the graph convolutional network model takes in two hyperparameters for this implementation. The first is the number of hidden graph convolution layers, and the second is the layer size. After each convolution, the node features are aggregated with the graph neighbours to create a new node feature with the size of the hidden layer size. Each time the graph convolution operation is performed, information from nodes farther along the graph is aggregated on the node and incorporated into the hidden layers. The number of hidden layers hyperparameter will control how far the node embeddings incorporate information along the graph, and the hidden layer size controls how much information is stored in the embeddings.

5.3 Results

Once the optimal hyperparameters are found, the the training and testing datasets are combined to train the final GCN model with the optimal hyperparameters, and the model is evaluated on the validation hold out set. The optimal hyperparameters and the results of the model applied to the validation set are detailed in table 1. Figure 4 shows the results of the model over the training lifetime. Figure 4.a shows the cross entropy loss of the GCN model being trained on the combined training and test partitions, and being validated on the validation set. The model quickly learns to minimise the loss function and has found a stable minimum after 100 epochs of training. Figure 4.b shows the error of the model and compares it to the target error derived from the $\phi_{a_d}^*$ state abstraction. The error is averaged over each sample in the validation set and a 95% confidence interval is shaded around the average. The GCN model quickly converges to an error of 0.13, compared to a target of 0.03. Figure 4.c shows the recall score measured as an average of the recall scores calculated on the validation set. The recall score rises quickly before plateauing at a value of 0.75.

Table 1: GCN model optimal hyperparameters and results

Weight Spread	10	Hidden Layers	1	Cross Entropy	0.6796	Recall	0.7457
Learning Rate	0.0977	Hidden Layer Size	57	Gap	0.2133	Gap $\phi_{a_d}^*$	0.0493
Weight Decay	0.0010	Epochs	500	Error	0.1347	Error $\phi_{a_d}^*$	0.0329

5.4 Discussion

The GCN model is able to approach the performance of the $\phi_{a_d}^*$ state abstraction, but plateaus at a gap that is twice the target gap. The graph convolutional network approach far outperforms the traditional machine learning approaches when measured as a state abstraction function. Though the model is trained using the cross entropy loss, which focuses on classification performance, the hyperparameter tuning was done to set up the model to minimise the gap. By minimising the gap during hyperparameter tuning, the model is set up to identify the sensitive states and prioritize those while learning from the cross entropy loss.

Initially, only the gap was used during hyperparameter tuning, however this method produces poor recall scores after the final training. The model was essentially finding new state abstraction functions which performed well, likely capturing the sensitive states, and assigning other states to new abstract states. However, the error during these trials was similar to the error when optimizing for the recall score as well. This means essentially that there is a fundamental limit to the GCN model for learning on MDP’s structures as graphs with purely node features. The reason for this is that it was realised far into the project that the underlying GCN model and the implementation in PyTorch geometric does not make use of edge features, meaning that information like transition probabilities cannot be encoded on the edges of the graph. In order to represent the MDP with edge features and make use of those features, an alternative graph neural network model will need to be used, one which incorporates edge features in the model architecture.

In practice, the model is still able to achieve a 0.13 error, meaning that while it does not perform as well as the $\phi_{a_d}^*$ state abstraction, it is still able to function as a good state abstraction function. Domain experts in conservation teams, or other domains, might still prefer to make the trade off between the computation time of the $\phi_{a_d}^*$ state abstraction and the accuracy of the GCN model, particularly if they find that it is able to correctly assign sensitive states to appropriate state abstractions.

6 Conclusion

Markov decision processes are very useful models with applications in reinforcement learning, computational decision planning and conservation science. In human operated domains, the optimal policy’s can be difficult to interpret by domain experts due to the high number of states that are needed to model certain situations. A solution found by Ferrer-Mestres et al., 2020 involves finding a state abstraction by defining a maximum number of abstract states, and constructing a K-MDP. While K-MDPs are easier to interpret, they rely on the optimal policy of the original MDP which can be computationally complex to solve.

This project aimed to circumvent the requirement that the original MDP needed to be solved first before state abstraction’s could be found. To do this I focused on using traditional and state of the art machine learning techniques to predict the abstract states purely from the structure of the MDP. By treating the abstract states as classes, the problem was set up as a multiclass classification problem.

Traditional machine learning, while able to maximise the accuracy of the predicted states, could not account for sensitive states, states that when misclassified have a large impact on the effectiveness of the K-MDP. This is because traditional methods assign equal value to each of the states. Since MDPs can be well represented by graph structures, a state of the art technique called a graph convolutional network was used to solve the problem instead. The GCN model was able to learn to correctly classify sensitive states and act as a good state abstraction function, but was unable to fully learn the state abstraction function used in the original work on K-MDPs. This is because GCN’s do not make use of edge features on the graph,

which limits their ability to learn the importance of the relationships between the different states.

Future work will include making three modifications to the graph neural network approach. The first will be to use a graph neural network which is able to learn from edge features, the second will be to construct a custom loss function from the gap and use this to train a graph neural network model that can be used as a state abstraction function, and the third will be to test the graph neural network approach on larger MDPs.

References

- Abel, D., Hershkowitz, D. E., & Littman, M. L. (2017). Near Optimal Behavior via Approximate State Abstraction [arXiv:1701.04113 [cs]]. Retrieved September 21, 2022, from <http://arxiv.org/abs/1701.04113>
- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Chadès, I., Chapron, G., Cros, M.-J., Garcia, F., & Sabbadin, R. (2014). MDPtoolbox: A multi-platform toolbox to solve stochastic dynamic programming problems. *Ecography*, 37(9), 916–920. <https://doi.org/10.1111/ecog.00888>
- Chadès, I., Curtis, J. M. R., & Martin, T. G. (2012). Setting realistic recovery targets for two interacting endangered species, sea otter and northern abalone. *Conservation Biology*, 26(6), 1016–1025. <https://doi.org/https://doi.org/10.1111/j.1523-1739.2012.01951.x>
- Costello, C., & Polasky, S. (2004). Dynamic reserve site selection. *Resource and Energy Economics*, 26(2), 157–174. <https://doi.org/10.1016/j.reseneeco.2003.11.005>
- Dazeley, R., Vamplew, P., & Cruz, F. (2023). Explainable reinforcement learning for broad-XAI: A conceptual framework and survey. *Neural Computing and Applications*. <https://doi.org/10.1007/s00521-023-08423-1>
- Ferrer-Mestres, J., Dietterich, T. G., Buffet, O., & Chadès, I. (2020). Solving K-MDPs. *Proceedings of the International Conference on Automated Planning and Scheduling*, 30, 110–118. Retrieved July 29, 2022, from <https://ojs.aaai.org/index.php/ICAPS/article/view/6651>
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Pedreschi, D., & Giannotti, F. (2018). A Survey Of Methods For Explaining Black Box Models [arXiv:1802.01933 [cs]]. <https://doi.org/10.48550/arXiv.1802.01933>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). Neural Networks. In T. Hastie, R. Tibshirani, & J. Friedman (Eds.), *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (pp. 389–416). Springer. https://doi.org/10.1007/978-0-387-84858-7_11
- Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2), 99–134. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X)
- Kingma, D. P., & Ba, J. (2017). Adam: A Method for Stochastic Optimization [arXiv:1412.6980 [cs]]. Retrieved June 10, 2023, from <http://arxiv.org/abs/1412.6980>
- Kipf, T. N. (2020). *Deep learning with graph-structured representations*. Retrieved May 31, 2023, from <https://dare.uva.nl/search?identifier=1b63b965-24c4-4bcd-aabb-b849056fa76d>
- Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks [arXiv:1609.02907 [cs, stat]]. Retrieved May 31, 2023, from <http://arxiv.org/abs/1609.02907>
- Li, L., Walsh, T. J., & Littman, M. L. (2006). Towards a Unified Theory of State Abstraction for MDPs, 10.
- Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization [arXiv:1711.05101 [cs, math]]. Retrieved June 10, 2023, from <http://arxiv.org/abs/1711.05101>

- Marescot, L., Chapron, G., Chadès, I., Fackler, P. L., Duchamp, C., Marboutin, E., & Gimenez, O. (2013). Complex decisions made simple: A primer on stochastic dynamic programming (R. Freckleton, Ed.). *Methods in Ecology and Evolution*, 4 (9), 872–884. <https://doi.org/10.1111/2041-210X.12082>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems 32* (pp. 8024–8035). Curran Associates, Inc. <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Petrik, M., & Luss, R. (2016). Interpretable Policies for Dynamic Product Recommendations.
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming* (1st ed.). Wiley. <https://doi.org/10.1002/9780470316887>
- Rosenblatt, F. (1961). *Principles of neurodynamics: Perceptions and the theory of brain mechanism*. Spartan Books.
- Sabbadin, R., Spring, D., & Rabier, C.-E. (2007). Dynamic reserve site selection under contagion risk of deforestation. *Ecological Modelling*, 201 (1), 75–81. <https://doi.org/10.1016/j.ecolmodel.2006.07.036>
- Sutton, R. S., & Barto, A. G. (1992). Reinforcement Learning: An Introduction, 352.
- Williams, R. J., & Baird, L. (1993). Tight Performance Bounds on Greedy Policies Based on Imperfect Value Functions. Retrieved May 28, 2023, from <https://www.semanticscholar.org/paper/Tight-Performance-Bounds-on-Greedy-Policies-Based-Williams-Baird>